

MEMBER 1: ML & AI LEAD - VIBE CODING ROADMAP

Smart India Hackathon 2026 | Team ILLUMINATI (ID: 27113) | PS: SIH1642

Primary Folder: ml/ | Dedicated Branch: feature/ml-model | Deliverables: 5 Files

PHASE 1: HISTORICAL BASELINE ENGINE

Prompt 1.1

Create ml/baseline.py. Define a BaselineCalculator class with an `__init__` method taking a Pandas DataFrame of clean observations (location, month, temperature, relative_humidity, pressure, wind_speed, rainfall). Add docstrings and type hints.

Prompt 1.2

In BaselineCalculator, add `compute_baselines()`. Group data by [location, month]. For each of the 5 weather variables, compute mean (mu), std dev (sigma), median, 25th percentile, 75th percentile, IQR, 5th percentile, and 95th percentile.

Prompt 1.3

In BaselineCalculator, handle zero standard deviations: set a minimum variance threshold (`sigma_min = 0.1`) to prevent division by zero during Z-score calculations.

Prompt 1.4

In BaselineCalculator, implement `save_to_json(filepath)` and `load_from_json(filepath)`. Save precomputed baselines to `data/processed/baseline_statistics.json` for O(1) in-memory lookup (<1ms).

Prompt 1.5

Add a helper `get_baseline(location: str, month: int) -> dict` in ml/baseline.py that returns baseline stats for a given location and month, raising a clean `ValueError` if location is unknown.

PHASE 2: STATISTICAL ANOMALY LAYER

Prompt 1.6

Create ml/anomaly_engine.py. Define `StatisticalEngine` class that takes baseline statistics from BaselineCalculator. Implement `calculate_z_scores(obs: dict, base: dict) -> dict` computing $Z = (X - \mu) / \sigma$.

Prompt 1.7

In `StatisticalEngine`, implement `compute_statistical_anomaly_score(z_scores: dict) -> float`. Use a normalized Euclidean/Mahalanobis-style norm of Z-scores mapped via a sigmoid/tanh function to scale [0.00, 1.00].

Prompt 1.8

Add individual variable departure flags in `StatisticalEngine`: flag any variable as primary departure if absolute Z-score $|Z| \geq 2.5$ ($p < 0.01$ significance).

PHASE 3: MACHINE LEARNING LAYER (ISOLATION FOREST)

MEMBER 1: ML & AI LEAD - VIBE CODING ROADMAP

Smart India Hackathon 2026 | Team ILLUMINATI (ID: 27113) | PS: SIH1642

Primary Folder: ml/ | Dedicated Branch: feature/ml-model | Deliverables: 5 Files

Prompt 1.9

In ml/anomaly_engine.py, create MLAnomalyEngine class. Initialize with Scikit-learn's IsolationForest(n_estimators=150, contamination=0.03, random_state=42, n_jobs=-1).

Prompt 1.10

In MLAnomalyEngine, implement prepare_features(df: pd.DataFrame) -> np.ndarray. Standardize feature vector combining raw metrics, baseline Z-deviations, cyclical temporal sin/cos features, and rolling statistics using StandardScaler.

Prompt 1.11

In MLAnomalyEngine, implement fit(X: np.ndarray). Fit the StandardScaler and IsolationForest. Ensure random seeds are fixed for 100% reproducible training.

Prompt 1.12

In MLAnomalyEngine, implement predict_raw_score(X: np.ndarray) -> float using decision_function(). Transform negative decision values into a continuous calibrated score [0.00, 1.00] where higher values indicate stronger anomalies.

PHASE 4: DUAL-ENGINE FUSION & SEVERITY MAPPING

Prompt 1.13

In ml/anomaly_engine.py, create DualAnomalyEngine fusing StatisticalEngine and MLAnomalyEngine: FinalScore = 0.4 * StatScore + 0.6 * MLScore. Ensure output is strictly bounded in [0.00, 1.00].

Prompt 1.14

In DualAnomalyEngine, implement classify_severity(score: float) -> str: map 0.00-0.39 to NORMAL, 0.40-0.69 to WATCH, 0.70-0.89 to HIGH, and 0.90-1.00 to CRITICAL.

Prompt 1.15

In DualAnomalyEngine, implement classify_anomaly_type(obs, base, z_scores) -> str: identify Heat Wave, Cold Wave, Extreme Rainfall, Severe Drought, Deep Depression, or Compound Weather Anomaly.

PHASE 5: EXPLAINABILITY & SERIALIZATION

Prompt 1.16

Create ml/explainability.py. Define ExplainabilityEngine. Implement compute_feature_contributions(z_scores: dict) -> list[dict] calculating relative percentage: $Contrib_i = (|Z_i| / \sum(|Z_j|)) * 100$. Rank features in descending order.

MEMBER 1: ML & AI LEAD - VIBE CODING ROADMAP

Smart India Hackathon 2026 | Team ILLUMINATI (ID: 27113) | PS: SIH1642

Primary Folder: ml/ | Dedicated Branch: feature/ml-model | Deliverables: 5 Files

Prompt 1.17

In ExplainabilityEngine, format each contributor object to include: feature, contribution_pct, observed, expected, unit, and direction ('HIGH' or 'LOW').

Prompt 1.18

In ExplainabilityEngine, implement generate_natural_language_summary(severity, anomaly_type, contributors) -> str producing clear diagnostic rationale sentences for disaster managers.

Prompt 1.19

Create ml/train.py. Write an end-to-end training pipeline loading data/processed/weather_cleaned.csv, computing baselines, training Isolation Forest, evaluating extreme scenarios, and saving artifacts to ml/models/isolation_forest.joblib and scaler.joblib.

Prompt 1.20

In ml/train.py, add self-validation: test a normal observation (assert score < 0.40) and cloudburst (assert score > 0.90). Print validation status and inference latency (must be <15ms).